The Recurring Bills Manager App helps users track and manage their recurring bills (e.g., electricity, water, rent, internet, etc.). The app allows users to add, delete, and update all bills, as well as mark them as Paid or Unpaid. All data must be stored locally using SQLite or AsyncStorage and the app must run on both Android and iOS via Expo.

Data Structure
Each bill includes the following attributes:

Field	Type	Description
`id`	integer	Unique identifier (auto-incremented)
`bill_name`	text	Bill name (e.g., "Electricity - September")
`category`	text	Type of bill (e.g., "Utilities", "Subscription", "Rent", "Finance", "Health")
`amount`	real	Amount in VND (e.g., 850.000)
`status`	text	Either "Paid" or "Unpaid"
`due_date`	text	Due date in ISO format (e.g., "2025-10-30")

Requirements 1: Bills List Screen (Home Screen)

Create the main screen displaying all recurring bills *(see **Figure 1**)*. This screen must include:

- Display bills in a **FlatList** showing: Bill Name, Category, Amount, Due Date, and Status. (0.5 points)
- Use **Bottom Tab Navigator** to switch between screens.
- On first launch, the list may be empty. Users can add new bills using a button.
- Show a **banner image** at the top of the Home screen.
- Tapping a bill navigates to its **Update screen**.

Task 2: Add New Bill Screen

Create a screen to add new bills (*see **Figure 3***):

- Input fields: Bill Name, Category, Amount, Due Date, and Status.

Note: The id field is auto-generated, and status defaults to "**Unpaid**".

- Save the new bill to Database and immediately update the Home screen list.
- Pressing the **Back** button should return to the Home screen.

Requirements 3: Update Bill Screen

- Create a screen for updating an existing bill. Allow users to **modify** Bill Name, Category, Amount, Due Date, or Status, and **save changes** to Database *(see **Figure 4**)*:
- Display the selected bill's current information in editable input fields.

- Include three buttons on the same row:
  - Update: Save modified data and return to Home screen.
  - Delete: Permanently remove the bill.
  - Back: Return to Home screen

Requirements 4: Unpaid Bills Summary Screen

Create a screen that displays only bills with status = "Unpaid" and shows a summary *(see **Figure 5**)*:

- Display all unpaid bills using a FlatList (showing name, category, and amount).
- Calculate and display at the bottom: the total outstanding amount (sum of all unpaid bills).

Requirements 5: Additional Requirements

- Validate input fields and provide immediate feedback for adding, updating, and deleting bills, including confirmation before deletion (*see **Figure 2***).
- UI/UX: Clean, modern interface with consistent colors, icons (e.g., from @expo/vector-icons), and spacing.

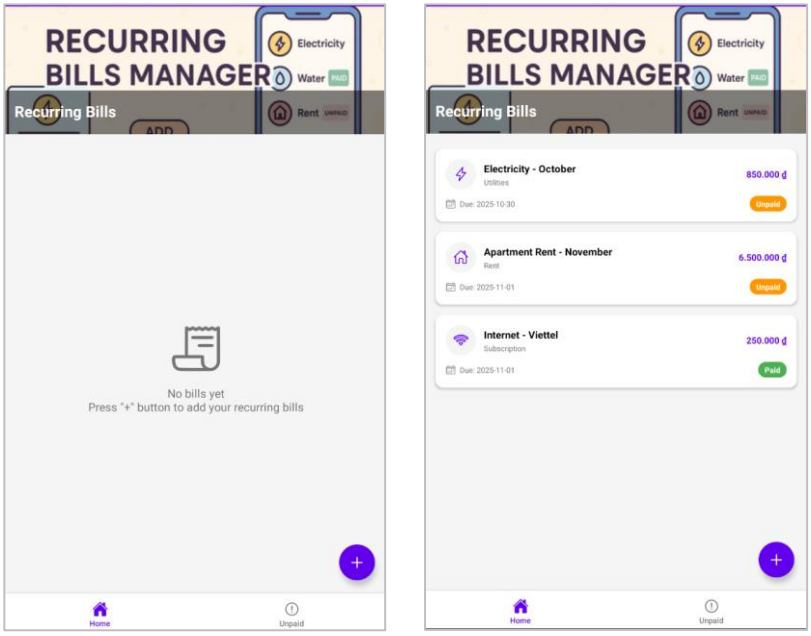


Figure 1: *Home Screen before and after adding recurring bills.*

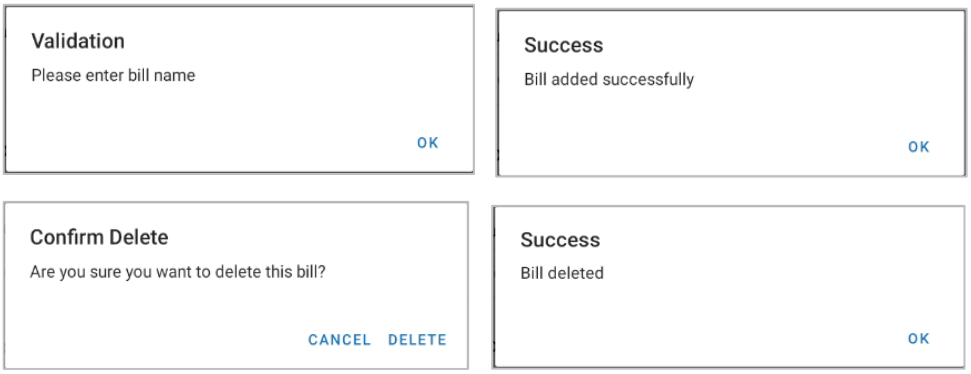


Figure 2: *Screens showing input validation (empty bill name), delete confirmation dialog, and success messages for add, update, and delete operations.*

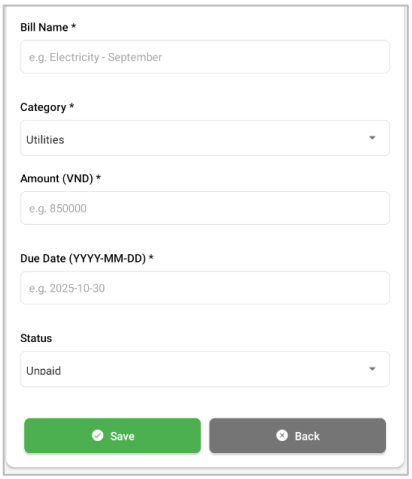


Figure 3: *Add Screen for creating a new recurring bill, with input fields, validation, and a submit button.*

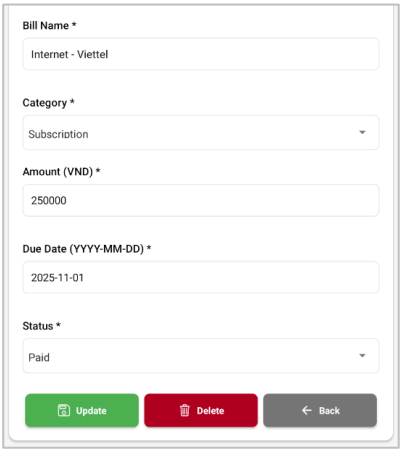


Figure 4: *Update Screen for editing bill information with fields for modification, and buttons to update or delete the bill.*

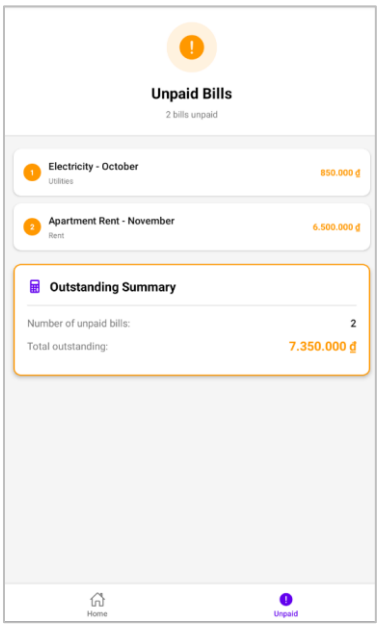


Figure 5: *Summary Screen – shows all unpaid bills and displays the total outstanding amount at the bottom.*